ERROR DETECTION & CORRECTION USING HAMMING CODE

AIM:

To write a program to implement error detection and correction using the Hamming code.

ALGORITHM:

- Start the program
- Read the 4-bit data bits
- Place data bits in positions 3, 5, 6, 7
- Calculate parity bits P_1, P_2, P_4 using XOR
- Form the 7-bit Hamming code
- Introduce a single-bit error at the given position
- Calculate the syndrome c1, c2, c4 & find the error position
- If an error exists, flip that bit to correct it
- Extract the original 4 data bits
- Stop the program

CODE:

```
def encode(d):
    h = [0] * 7
    h[2], h[4], h[5], h[6] = d
    h[0] = h[2] ^ h[4] ^ h[6]
    h[1] = h[2] ^ h[5] ^ h[6]
    h[3] = h[4] ^ h[5] ^ h[6]
    return h
```

```
=== HAMMING CODE - ERROR DETECTION AND CORRECTION ===

Enter number of 4-bit data blocks in the stream: 4

--- Block 1 ---
Enter 4 data bits (space separated, e.g. 1 0 1 1): 1 0 1 1
Original Data Bits       : 1011
Encoded Hamming Code (7 bits): 0110011
Enter bit position to introduce error (1-7, 0 = no error): 5
Transmitted Code (with error induced at pos 5): 0110111
Received Code            : 0110111
Checking for errors...
 Error detected at bit position: 5 -> Correcting...
Corrected Code           : 0110011
Recovered Data Bits      : 1011

--- Block 2 ---
Enter 4 data bits (space separated, e.g. 1 0 1 1): 1 1 0 1
Original Data Bits       : 1101
Encoded Hamming Code (7 bits): 1010101
Enter bit position to introduce error (1-7, 0 = no error): 6
Transmitted Code (with error induced at pos 6): 1010111
Received Code            : 1010111
Checking for errors...
 Error detected at bit position: 6 -> Correcting...
Corrected Code           : 1010101
Recovered Data Bits      : 1101

--- Block 3 ---
Enter 4 data bits (space separated, e.g. 1 0 1 1): 1 1 1 0
Original Data Bits       : 1110

Encoded Hamming Code (7 bits): 0010110
Enter bit position to introduce error (1-7, 0 = no error): 7
Transmitted Code (with error induced at pos 7): 0010111
Received Code            : 0010111
Checking for errors...
 Error detected at bit position: 7 -> Correcting...
Corrected Code           : 0010110
Recovered Data Bits      : 1110

--- Block 4 ---
Enter 4 data bits (space separated, e.g. 1 0 1 1): 0 1 1 1
Original Data Bits       : 0111
Encoded Hamming Code (7 bits): 0001111
Enter bit position to introduce error (1-7, 0 = no error): 8
Transmitted Code (no error induced): 0001111
Received Code            : 0001111
Checking for errors...
 No error detected.
Corrected Code           : 0001111
Recovered Data Bits      : 0111
```

```
def detect (h):
    e = (h[0] ^ h[2] ^ h[4] ^ h[6]) + 1
        2 * (h[1] ^ h[2] ^ h[5] ^ h[6]) + 1
        4 * (h[3] ^ h[4] ^ h[5] ^ h[6])

    if e:
        print ("Error at position:", e)
        h[e-1] ^= 1

    else:
        print ("No error")
    return h

n = int (input ("Blocks:"))
for i in range (n):
        d = list (map (int, input ("Enter 4 bits:").split()))
        h = encode (d)
        print ("Encoded:", *h)
        p = int (input ("Error position (0-7):"))
        if 1 <= p <= 7:
            h[p-1] ^= 1
        print ("Received:", *h)
        detect (h)
        print ("Corrected:", *h)
        print ("Recovered:", h[2], h[4], h[5], h[6])
```

RESULT:

Thus, the Hamming code program was successfully implemented the detect and correct a single-bit error & recover the original data bits.